

Problem 2 (Uphill paths), Cell 4: $U(Q_7)$ and $U(Q_8)$

Team submission

BAINSA Hackathon, 26 September 2026

1 Cell 4

Official setting. As in Cell 1 (§A.1).

Task. Determine $U(Q_7)$ and $U(Q_8)$, and give a labelling attaining each value.

Status. Solved. The proof is by hand from $\nabla(Q_6) = 28$ (Cell 3). That fact is computer-assisted, so this cell inherits its computation (Lemma C.1) and needs no new one.

Answer. $U(Q_7) = 464$ and $U(Q_8) = 1040$; $\nabla(Q_7) = 56$ and $\nabla(Q_8) = 112$ (Theorem 1.2). The labellings are in §1.3.

1.1 Doubling

Lemma 1.1 (Doubling). $\nabla(Q_d) \geq 2\nabla(Q_{d-1})$ for every $d \geq 2$.

Proof. Fixing the last coordinate splits Q_d into two disjoint induced copies of Q_{d-1} . If R is a decycling set of Q_d , then the part of R in each copy is a decycling set of that copy: an induced subgraph of a forest is a forest. So each copy contains at least $\nabla(Q_{d-1})$ vertices of R . $\square$

Theorem 1.2. $\nabla(Q_7) = 56$, $\nabla(Q_8) = 112$, $U(Q_7) = 464$ and $U(Q_8) = 1040$.

Proof. Lower bounds: by Lemma C.1 and Lemma 1.1, $\nabla(Q_7) \geq 2 \cdot 28 = 56$ and $\nabla(Q_8) \geq 2 \cdot 56 = 112$. By Theorem A.4, $U(Q_7) \geq 128 + 6 \cdot 56 = 464$ and $U(Q_8) \geq 256 + 7 \cdot 112 = 1040$.

Upper bounds: let $C_7 \subseteq \{0,1\}^7$ be the linear code spanned by 1111000, 1100110 and 1010101. Its 7 nonzero words are the sums of nonempty subsets of the generators:

$$1111000, 1100110, 1010101, 0011110, 0101101, 0110011, 1001011.$$

All have weight 4. So $C_7 \subseteq E_7$, $|C_7| = 8$ and all distances are 4 (this is the simplex code, i.e. the dual of the Hamming code of length 7). Let $C_8 \subseteq \{0,1\}^8$ be the extended Hamming code, spanned by 11110000, 11001100, 10101010 and 11111111. Its 16 words are $C_7' \cup (C_7' \oplus 11111111)$, where C_7' is the set of words of C_7 with a 0 appended. The words of C_7' have weights 0 and 4, and their complements have weights 8 and 4. The distance between x and $y \oplus 11111111$, for $x, y \in C_7'$, is $8 - |x \oplus y| \in \{8, 4\}$. So $C_8 \subseteq E_8$ has minimum distance 4 and $|C_8| = 16$. Proposition A.6 gives decycling sets of sizes $64 - 8 = 56$ and $128 - 16 = 112$, and labellings with $128 + 6 \cdot 56 = 464$ and $256 + 7 \cdot 112 = 1040$ uphill paths. $\square$

1.2 Summary for $d \leq 8$

Corollary 1.3. For $2 \leq d \leq 8$, $U(Q_d) = 2^d + (d-1)\nabla(Q_d) = 2^d + (d-1)(2^{d-1} - K_d)$, where K_d is the size of the code used:

d	2	3	4	5	6	7	8
K_d	1	1	2	2	4	8	16
$\nabla(Q_d)$	1	3	6	14	28	56	112
$U(Q_d)$	5	14	34	88	204	464	1040
counting bound $\lceil \frac{(d-2)2^{d-1}+1}{d-1} \rceil$	1	3	6	13	26	54	110

In every case the optimum is attained by a code labelling L_C , and the set R_f of expensive vertices is one parity class minus a code.

Proof. $d = 2$ and $d = 3, 4$: Cell 1. $d = 5$: Cell 2. $d = 6$: Cell 3. $d = 7, 8$: Theorem 1.2. $\square$

Remark 1.4. The values $K_d = 1, 1, 2, 2, 4, 8, 16$ are the maximum sizes $A(d-1, 3)$ of binary codes of length $d-1$ and minimum distance 3. Adding a parity bit turns such a code into a code in E_d with minimum distance 4, and conversely. The proofs above do not use this. For $d = 9$ the same formula would give $\nabla(Q_9) \leq 256 - A(8, 3) = 236$, because $A(8, 3) = 20$ [4]. This is the starting point of Cells 5 and 6.

1.3 The labellings

In both lists: the codewords, then all odd vertices, then the remaining even vertices, each block in increasing binary order. The counts (464 and 1040) were checked with `code/uphill_eval.py`.

Q_7 , $C = C_7$ (8 codewords), 464 uphill paths:

```

1: 0000000 0011110 0101101 0110011 1001011 1010101 1100110 1111000
9: 0000001 0000010 0000100 0000111 0001000 0001011 0001101 0001110
17: 0010000 0010011 0010101 0010110 0011001 0011010 0011100 0011111
25: 0100000 0100011 0100101 0100110 0101001 0101010 0101100 0101111
33: 0110001 0110010 0110100 0110111 0111000 0111011 0111101 0111110
41: 1000000 1000011 1000101 1000110 1001001 1001010 1001100 1001111
49: 1010001 1010010 1010100 1010111 1011000 1011011 1011101 1011110
57: 1100001 1100010 1100100 1100111 1101000 1101011 1101101 1101110
65: 1110000 1110011 1110101 1110110 1111001 1111010 1111100 1111111
73: 0000011 0000101 0000110 0001001 0001010 0001100 0001111 0010001
81: 0010010 0010100 0010111 0011000 0011011 0011101 0100001 0100010
89: 0100100 0100111 0101000 0101011 0101110 0110000 0110101 0110110
97: 0111001 0111010 0111100 0111111 1000001 1000010 1000100 1000111
105: 1001000 1001101 1001110 1010000 1010011 1010110 1011001 1011010
113: 1011100 1011111 1100000 1100011 1100101 1101001 1101010 1101100
121: 1101111 1110001 1110010 1110100 1110111 1111011 1111101 1111110

```

Q_8 , $C = C_8$ (16 codewords), 1040 uphill paths:

```

1: 00000000 00001111 00110011 00111100 01010101 01011010 01100110 01101001 10010110
10: 10011001 10100101 10101010 11000011 11001100 11110000 11111111 00000001 00000010
19: 00000100 00000111 00001000 00001011 00001101 00001110 00010000 00010011 00010101
28: 00010110 00011001 00011010 00011100 00011111 00100000 00100011 00100101 00100110
37: 00101001 00101010 00101100 00101111 00110001 00110010 00110100 00110111 00111000
46: 00111011 00111101 00111110 01000000 01000011 01000101 01000110 01001001 01001010
55: 01001100 01001111 01010001 01010010 01010100 01010111 01011000 01011011 01011101
64: 01011110 01100001 01100010 01100100 01100111 01101000 01101011 01101101 01101110
73: 01110000 01110011 01110101 01110110 01111001 01111010 01111100 01111111 10000000
82: 10000011 10000101 10000110 10001001 10001010 10001100 10001111 10010001 10010010
91: 10010100 10010111 10011000 10011011 10011101 10011110 10100001 10100010 10100100
100: 10100111 10101000 10101011 10101101 10101110 10110000 10110011 10110101 10110110
109: 10111001 10111010 10111100 10111111 11000001 11000010 11000100 11000111 11001000
118: 11001011 11001101 11001110 11010000 11010011 11010101 11010110 11011001 11011010
127: 11011100 11011111 11100000 11100011 11100101 11100110 11101001 11101010 11101100
136: 11101111 11110001 11110010 11110100 11110111 11111000 11111011 11111101 11111110
145: 00000011 00000101 00000110 00001001 00001010 00001100 00010001 00010010 00010100
154: 00010111 00011000 00011011 00011101 00011110 00100001 00100010 00100100 00100111
163: 00101000 00101011 00101101 00101110 00110000 00110101 00110110 00111001 00111010

```

```

172: 00111111 01000001 01000010 01000100 01000111 01001000 01001011 01001101 01001110
181: 01010000 01010011 01010110 01011001 01011100 01011111 01100000 01100011 01100101
190: 01101010 01101100 01101111 01110001 01110010 01110100 01110111 01111000 01111011
199: 01111101 01111110 10000001 10000010 10000100 10000111 10001000 10001011 10001101
208: 10001110 10010000 10010011 10010101 10011010 10011100 10011111 10100000 10100011
217: 10100110 10101001 10101100 10101111 10110001 10110010 10110100 10110111 10111000
226: 10111011 10111101 10111110 11000000 11000101 11000110 11001001 11001010 11001111
235: 11010001 11010010 11010100 11010111 11011000 11011011 11011101 11011110 11100001
244: 11100010 11100100 11100111 11101000 11101011 11101101 11101110 11110011 11110101
253: 11110110 11111001 11111010 11111100

```

A Results used: Cell 1 ($U(Q_3)$ and $U(Q_4)$), as handed in for that cell

Official setting.

Let G be a finite simple graph on n vertices. A *labelling* of G is a bijection f from $V(G)$ to $\{1, 2, \dots, n\}$. Fix a labelling. A vertex v is a *valley* if every neighbour w of v has $f(w) > f(v)$; an isolated vertex counts as a valley. An *uphill path* is a sequence $(v_1, v_2, \dots, v_k)$ with $k \geq 1$ in which v_1 is a valley, v_i and v_{i+1} are adjacent for each i , and $f(v_1) < f(v_2) < \dots < f(v_k)$. A valley on its own ($k = 1$) counts as an uphill path. Let $U(G)$ be the smallest possible number of uphill paths, taken over all labellings of G . (IMO 2022 Problem 6 asks for U of the $n \times n$ grid graph; the answer is $2n^2 - 2n + 1$.) The d -dimensional hypercube Q_d has vertex set $\{0, 1\}^d$, and two vertices are adjacent when they differ in exactly one coordinate; Q_d has 2^d vertices and $d \cdot 2^{d-1}$ edges. For C1 to C4, hand in the values and, for each value, an explicit labelling that attains it, as a list of the 2^d vertices in increasing label order, written as 0/1 strings.

Task. Determine $U(Q_3)$ and $U(Q_4)$, and give a labelling attaining each value.

Status. **Solved**, by hand. No computation is needed; the labellings were also counted mechanically (code/uphill_eval.py).

Answer and where it is proved.

- $U(Q_3) = 14$ and $U(Q_4) = 34$ (Theorem A.8); labellings in §A.8.
- The proof rests on an exact formula for the number of uphill paths (Proposition A.3). It gives $U(Q_d) \geq 2^d + (d-1)\nabla(Q_d)$ for every d , where ∇ is the decycling number (Theorem A.4). In the other direction, every even-weight code with minimum distance 4 gives a labelling (Proposition A.6).
- Sections A.1–A.6 hold for every d and are used by all the later cells of this problem.

A.1 Counting uphill paths

Fix a graph G and a labelling f . A *lower neighbour* of v is a neighbour w with $f(w) < f(v)$, and a *higher neighbour* is a neighbour w with $f(w) > f(v)$. Write $\text{out}(v)$ for the number of higher neighbours of v . So v is a valley if and only if it has no lower neighbour. Let $N(v) = N_f(v)$ be the number of uphill paths whose last vertex is v , and $P(f) = \sum_v N(v)$ the number of uphill paths. Then $U(G) = \min_f P(f)$.

Lemma A.1 (Recursion). $N(v) = 1$ if v is a valley, and $N(v) = \sum_w N(w)$, summed over the lower neighbours w of v , otherwise. In particular $N(v) \geq 1$ for every v .

Proof. If $(v_1, \dots, v_k)$ is an uphill path with $v_k = v$ and $k \geq 2$, then v_{k-1} is a lower neighbour of v and $(v_1, \dots, v_{k-1})$ is an uphill path ending at v_{k-1} . Conversely, if w is a lower neighbour of v , then every uphill path ending at w extends by v to an uphill path ending at v . These two maps are inverse to each other. The one-vertex sequence (v) is an uphill path if and only if v is a valley, and a valley has no lower neighbours. The inequality $N(v) \geq 1$ follows by induction on $f(v)$: a non-valley has at least one lower neighbour. $\square$

A.2 Vertices with a single uphill path

Put $S = S_f = \{v : N(v) = 1\}$ and $R = R_f = \{v : N(v) \geq 2\}$. Write $G[X]$ for the subgraph induced by X , and $e(X)$ for its number of edges.

Lemma A.2. (a) *If w is a lower neighbour of v , then $N(v) \geq N(w)$. A vertex of S that is not a valley has exactly one lower neighbour, and it lies in S .*

(b) *Every edge between $s \in S$ and $u \in R$ satisfies $f(s) < f(u)$. Every higher neighbour of a vertex of R lies in R .*

(c) *$G[S]$ is a forest. Every valley of G lies in S , and every component of $G[S]$ contains exactly one valley.*

Proof. (a) By Lemma A.1, $N(v)$ is a sum of terms ≥ 1 that includes $N(w)$. If $v \in S$ is not a valley, then this sum equals 1, so it has a single term, equal to 1.

(b) If $f(u) < f(s)$, then u is a lower neighbour of s and (a) gives $N(s) \geq N(u) \geq 2$, a contradiction. Likewise, a higher neighbour v of $u \in R$ has $N(v) \geq N(u) \geq 2$.

(c) Valleys have $N = 1$. Every edge of $G[S]$ has a higher end v , and by (a) the lower end is the unique lower neighbour of v . So $e(S)$ equals the number of non-valleys in S . Let K be a component of $G[S]$, with n_K vertices. Its vertex of smallest label has no lower neighbour in K . By (a) it has no lower neighbour in G at all, so it is a valley. Counting as above inside K , $e(K) = n_K - (\text{number of valleys in } K) \leq n_K - 1$. A connected graph has $e(K) \geq n_K - 1$. Hence K is a tree and contains exactly one valley. $\square$

A.3 An exact formula

Proposition A.3 (Exact formula). *For every graph G and every labelling f ,*

$$P(f) = |V(G)| + \sum_{u \in R} (\deg u - 1) + \sum_{u \in R} (N(u) - 2) \text{out}(u).$$

All terms of the last sum are ≥ 0 . In particular, if G is d -regular, then $P(f) \geq |V(G)| + (d-1)|R_f|$.

Proof. Sum the recursion of Lemma A.1 over all v . Each pair $(w, \text{higher neighbour } v \text{ of } w)$ contributes $N(w)$ once, so

$$P(f) = \#\{\text{valleys}\} + \sum_w N(w) \text{out}(w).$$

Let c be the number of components of $G[S]$. By Lemma A.2(c), there are c valleys and $e(S) = |S| - c$. For $w \in S$ we have $N(w) = 1$. Moreover, $\sum_{w \in S} \text{out}(w) = e(S) + e(S, R)$, because each edge inside S is counted at its lower end, each S – R edge is counted at its end in S (Lemma A.2(b)), and edges inside R are not counted. For $w \in R$, all higher neighbours lie in R (Lemma A.2(b)), so $\sum_{w \in R} \text{out}(w) = e(R)$. Using $e(S, R) = \sum_{u \in R} \deg u - 2e(R)$, we get

$$P(f) = c + (|S| - c) + e(S, R) + \sum_{u \in R} N(u) \text{out}(u) = |S| + \sum_{u \in R} \deg u + \sum_{u \in R} (N(u) - 2) \text{out}(u),$$

and $|S| = |V(G)| - |R|$. Finally $N(u) \geq 2$ on R . $\square$

A.4 Decycling sets

A set $R \subseteq V(G)$ is a *decycling set* (or feedback vertex set) if $G - R = G[V \setminus R]$ is a forest. The *decycling number* $\nabla(G)$ is the smallest size of a decycling set. By Lemma A.2(c), R_f is a decycling set for every labelling f .

Theorem A.4 (Lower bound). *If G is d -regular, then $U(G) \geq |V(G)| + (d-1)\nabla(G)$. In particular, $U(Q_d) \geq 2^d + (d-1)\nabla(Q_d)$ for every $d \geq 1$.*

Proof. Proposition A.3 and $|R_f| \geq \nabla(G)$. □

Proposition A.5 (Upper bound). *Let G be d -regular and let R be a decycling set that is independent (no edge inside R). Then some labelling has exactly $|V(G)| + (d-1)|R|$ uphill paths. Consequently, if Q_d has an independent decycling set of size $\nabla(Q_d)$, then $U(Q_d) = 2^d + (d-1)\nabla(Q_d)$.*

Proof. Give the smallest labels to $V \setminus R$, tree by tree. Inside each tree of the forest $G - R$, use breadth-first order from a root. Then give the remaining labels to R , in any order. The neighbours of a vertex $v \notin R$ are its tree neighbours and some vertices of R , and the latter come later. In breadth-first order, a non-root vertex comes after its parent and before its children. So a root is a valley, and a non-root vertex has exactly one lower neighbour, its parent. By induction along the order, $N(v) = 1$ for all $v \notin R$. A vertex $u \in R$ has all its d neighbours outside R , all labelled earlier, so $N(u) = d$. The total is $(|V| - |R|) + d|R|$. The last sentence then follows from Theorem A.4. □

A.5 Code labellings

For $x \in \{0, 1\}^d$, let $|x|$ be its weight (number of ones). Let E_d and O_d be the sets of vertices of even and of odd weight. Every edge of Q_d joins E_d to O_d , so both sets are independent and have size 2^{d-1} . The distance $|x \oplus y|$ of two vertices of E_d is even.

Proposition A.6 (Code labelling). *Let $d \geq 2$, and let $C \subseteq E_d$ have pairwise distances ≥ 4 . Consider the labelling L_C that lists first the vertices of C , then all of O_d , then $E_d \setminus C$ (each block in any order). It has exactly*

$$P(L_C) = 2^d + (d-1)(2^{d-1} - |C|)$$

uphill paths. Moreover, $E_d \setminus C$ is an independent decycling set of Q_d . Hence $\nabla(Q_d) \leq 2^{d-1} - |C|$ and $U(Q_d) \leq 2^d + (d-1)(2^{d-1} - |C|)$.

Proof. We use Lemma A.1. A codeword has only odd neighbours, all labelled later, so it is a valley: $N = 1$. The earlier neighbours of an odd vertex x are exactly its neighbours in C . Two of them would be at distance 2 from each other, so there is at most one. Hence $N(x) = 1$: either x is a valley, or it has a single lower neighbour, with $N = 1$. A vertex $y \in E_d \setminus C$ has d odd neighbours, all labelled earlier, so $N(y) = d$. Summing, $P = |C| + 2^{d-1} + d(2^{d-1} - |C|)$. In this labelling $R_{L_C} = E_d \setminus C$ (as $d \geq 2$), which is decycling by Lemma A.2(c) and independent since it lies in E_d . □

A.6 A counting bound for the decycling number

Lemma A.7. *Let G be d -regular with n vertices and m edges, and let $R \neq V(G)$ be a decycling set. Let c be the number of components of $G - R$. Then*

$$(d-1)|R| = m - n + c + e(R) \geq m - n + 1.$$

For Q_d this reads $(d-1)|R| \geq (d-2)2^{d-1} + 1$.

Proof. $G - R$ is a forest with $n - |R|$ vertices and c components, so it has $n - |R| - c$ edges. The edges meeting R number $d|R| - e(R)$. Adding up gives $m = n - |R| - c + d|R| - e(R)$. □

A.7 Q_3 and Q_4

Theorem A.8. $U(Q_3) = 14$ and $U(Q_4) = 34$. Also $\nabla(Q_3) = 3$ and $\nabla(Q_4) = 6$.

Proof. $d = 3$: Lemma A.7 gives $2\nabla(Q_3) \geq 5$, so $\nabla(Q_3) \geq 3$, and Theorem A.4 gives $U(Q_3) \geq 8 + 2 \cdot 3 = 14$. The code $C = \{000\}$ in Proposition A.6 gives a labelling with $8 + 2 \cdot (4 - 1) = 14$ uphill paths and a decycling set of size 3.

$d = 4$: Lemma A.7 gives $3\nabla(Q_4) \geq 17$, so $\nabla(Q_4) \geq 6$ and $U(Q_4) \geq 16 + 3 \cdot 6 = 34$. The code $C = \{0000, 1111\}$ (distance 4) gives $16 + 3 \cdot (8 - 2) = 34$ and a decycling set of size 6. $\square$

Remark A.9. An exhaustive check over all $8! = 40\,320$ labellings of Q_3 (`uphill_eval.py --brute 3`) also returns 14. The proof above does not use it. For $d = 2$ the same argument gives $U(Q_2) = U(C_4) = 5$. In words, Proposition A.3 says that an optimal labelling must make few vertices “expensive” ($N \geq 2$), and those vertices must still break every cycle. For an arbitrary graph, the edge count in the proof of Lemma A.7 reads $|E| = |S| - c + \sum_{u \in R} \deg u - e(R)$. Combined with Proposition A.3, it gives $P(f) \geq |E| + c + e(R) \geq |E| + 1$. This is the lower bound $2n^2 - 2n + 1$ of the grid problem [3].

A.8 The labellings

Vertices in increasing label order (the number before each row is the label of its first entry). The labellings are L_C of Proposition A.6: first the codewords, then the odd vertices, then the remaining even vertices, each block in increasing binary order.

Q_3 , $C = \{000\}$, 14 uphill paths:

1: 000 001 010 100 111 011 101 110

Q_4 , $C = \{0000, 1111\}$, 34 uphill paths:

1: 0000 1111 0001 0010 0100 0111 1000 1011
9: 1101 1110 0011 0101 0110 1001 1010 1100

Both counts were confirmed with the exact evaluator `code/uphill_eval.py`, which applies Lemma A.1 directly.

B Results used: Cell 2 ($U(Q_5)$), as handed in for that cell

Official setting. As in Cell 1 (§A.1): $U(G)$ is the least number of uphill paths over all labellings of G . Values are handed in together with an explicit labelling, listed as the 2^d vertices in increasing label order.

Task. Determine $U(Q_5)$, and give a labelling attaining it.

Status. Solved. The upper bound is by hand. The lower bound is **computer-assisted**: one SAT computation, below one second, repeated with two solvers.

Answer. $U(Q_5) = 88$, and $\nabla(Q_5) = 14$ (Theorem B.2). The labelling is in §B.2.

B.1 The decycling number of Q_5

The counting bound of Lemma A.7 gives only $4\nabla(Q_5) \geq 3 \cdot 16 + 1$, that is, $\nabla(Q_5) \geq 13$, and so $U(Q_5) \geq 84$. This bound is not attained.

Lemma B.1 (computer-assisted). Q_5 has no decycling set with 13 or fewer vertices. Hence $\nabla(Q_5) \geq 14$.

Method, and why the computation proves the lemma. This is claim (L5) of `code/uphill_lower.py`. The variables are x_v ($v \in V(Q_5)$), with x_v meaning $v \in R$. The formula F contains:

- (a) a CNF encoding (sequential counter) of $\sum_v x_v \leq 13$;
- (b) for every 4-cycle K of Q_5 , the clause $\bigvee_{v \in K} x_v$;
- (c) the clauses $\bigvee_{v \in K} x_v$ for cycles K found during the loop below.

Each clause of type (b) or (c) says that R meets the cycle K , and every decycling set does this. So every decycling set of size at most 13 satisfies F at every stage.

Loop. The solver returns a model R , or reports UNSAT. If it returns R , we compute $Q_5 - R$ with `networkx`. If this graph has a cycle, we add the clauses of a cycle basis of every cyclic component (type (c)) and ask again. The new clauses are violated by the current model, so no model is returned twice. Hence the loop ends, after finitely many rounds. It ends in one of two ways: a model R for which $Q_5 - R$ is a forest (a decycling set of size ≤ 13), or UNSAT.

Result. UNSAT, after 348 rounds in 0.4s with CaDiCaL 1.5.3, and after 357 rounds in 0.3s with Glucose 4, both through `python-sat`. Since every decycling set of size ≤ 13 would be a model, none exists. The only facts used are the correctness of the cardinality encoding and of the SAT solvers. Running two independent solvers guards against a solver bug. $\square$

Theorem B.2. $\nabla(Q_5) = 14$ and $U(Q_5) = 88$.

Proof. Lower bound: Lemma B.1 and Theorem A.4 give $U(Q_5) \geq 32 + 4 \cdot 14 = 88$. Upper bound: $C = \{00000, 11110\}$ lies in E_5 and has distance 4. By Proposition A.6, $E_5 \setminus C$ is a decycling set of size $16 - 2 = 14$, and the labelling L_C has $32 + 4 \cdot 14 = 88$ uphill paths. $\square$

Remark B.3. A set $C \subseteq E_5$ with pairwise distances ≥ 4 has at most two elements. Suppose it had three. Translating by one of them (an even vector, which preserves E_5 and all distances), we may assume $C = \{0, a, b\}$. Then $|a| = |b| = 4$, so a and b are 4-subsets of a 5-set and meet in at least 3 coordinates. This gives $|a \oplus b| \leq 2$, a contradiction. So Proposition A.6 cannot give less than 88, and Lemma B.1 shows that no labelling can.

B.2 The labelling

L_C with $C = \{00000, 11110\}$: codewords, then the 16 odd vertices, then the remaining 14 even vertices, each block in increasing binary order. It has 88 uphill paths (checked with `code/uphill_eval.py`).

```

1: 00000 11110 00001 00010 00100 00111 01000 01011
9: 01101 01110 10000 10011 10101 10110 11001 11010
17: 11100 11111 00011 00101 00110 01001 01010 01100
25: 01111 10001 10010 10100 10111 11000 11011 11101

```

C Results used: Cell 3 ($U(Q_6)$), as handed in for that cell

Official setting. As in Cell 1 (§A.1).

Task. Determine $U(Q_6)$, and give a labelling attaining it.

Status. Solved. The upper bound is by hand. The lower bound is **computer-assisted**: one SAT computation, about 10s, repeated with two solvers.

Answer. $U(Q_6) = 204$, and $\nabla(Q_6) = 28$ (Theorem C.2). The labelling is in §C.2.

C.1 The decycling number of Q_6

Lemma A.7 gives $5\nabla(Q_6) \geq 4 \cdot 32 + 1$, that is, $\nabla(Q_6) \geq 26$. The true value is 28.

Lemma C.1 (computer-assisted). Q_6 has no decycling set with 27 or fewer vertices. Hence $\nabla(Q_6) \geq 28$.

Method. This is claim (L6) of `code/uphill_lower.py`. It uses the same counterexample-guided SAT loop as Lemma B.1, with $d = 6$ and bound $\sum_v x_v \leq 27$. The argument there shows that every decycling set of size ≤ 27 satisfies the formula at every stage, and that the loop ends. The result is UNSAT: after 685 rounds in about 10 s with CaDiCaL 1.5.3, and after 689 rounds in about 8 s with Glucose 4. $\square$

Theorem C.2. $\nabla(Q_6) = 28$ and $U(Q_6) = 204$.

Proof. Lower bound: by Lemma C.1 and Theorem A.4, $U(Q_6) \geq 64 + 5 \cdot 28 = 204$. Upper bound: let

$$C = \{000000, 111100, 110011, 001111\},$$

the linear code spanned by 111100 and 110011. All its nonzero words have weight 4, so its pairwise distances are 4. By Proposition A.6, $E_6 \setminus C$ is a decycling set of size $32 - 4 = 28$, and L_C has $64 + 5 \cdot 28 = 204$ uphill paths. $\square$

Remark C.3. Cell 5 needs a stronger fact: every decycling set of Q_6 of size 28 lies inside E_6 or inside O_6 . This is Lemma P6 of Cell 5, claim (P6) of the same script. It is stated and proved there, where it is used.

C.2 The labelling

L_C for the code C above: the 4 codewords, then the 32 odd vertices, then the remaining 28 even vertices, each block in increasing binary order. It has 204 uphill paths (checked with `code/uphill_eval.py`).

```

1: 000000 001111 110011 111100 000001 000010 000100 000111
9: 001000 001011 001101 001110 010000 010011 010101 010110
17: 011001 011010 011100 011111 100000 100011 100101 100110
25: 101001 101010 101100 101111 110001 110010 110100 110111
33: 111000 111011 111101 111110 000011 000101 000110 001001
41: 001010 001100 010001 010010 010100 010111 011000 011011
49: 011101 011110 100001 100010 100100 100111 101000 101011
57: 101101 101110 110000 110101 110110 111001 111010 111111

```

References

- [1] S. Bau, L. W. Beineke, The decycling number of graphs, *Australas. J. Combin.* 25 (2002) 285–298.
- [2] P. Focardi, F. L. Luccio, D. Peleg, Feedback vertex set in hypercubes, *Inform. Process. Lett.* 76 (2000) 1–5.
- [3] International Mathematical Olympiad 2022, Problem 6 (uphill paths in a Nordic square).
- [4] F. J. MacWilliams, N. J. A. Sloane, *The Theory of Error-Correcting Codes*, North-Holland, 1977.
- [5] D. A. Pike, Decycling hypercubes, *Graphs Combin.* 19 (2003) 547–550.